

Maximum Score From Grid Operations

 LeetCode 

 17479  Apr 24, 2026

Editorial

Approach: Dynamic Programming

Intuition

Unless otherwise specified, all matrix indices start from 1. That is, given a square matrix of dimension n , the top-left element has index $(1, 1)$ and is located in the first row and first column, while the bottom-right element has index (n, n) and is located in the n -th row and n -th column.

Start with a dynamic programming approach of $O(n^4)$

From the problem statement, we can observe that after any number of operations, each column must end up with black cells on top and white cells on the bottom. Formally, each column can be viewed as an array of length n , where the first i elements are black and the remaining $n - i$ elements are white, for some $0 \leq i \leq n$.

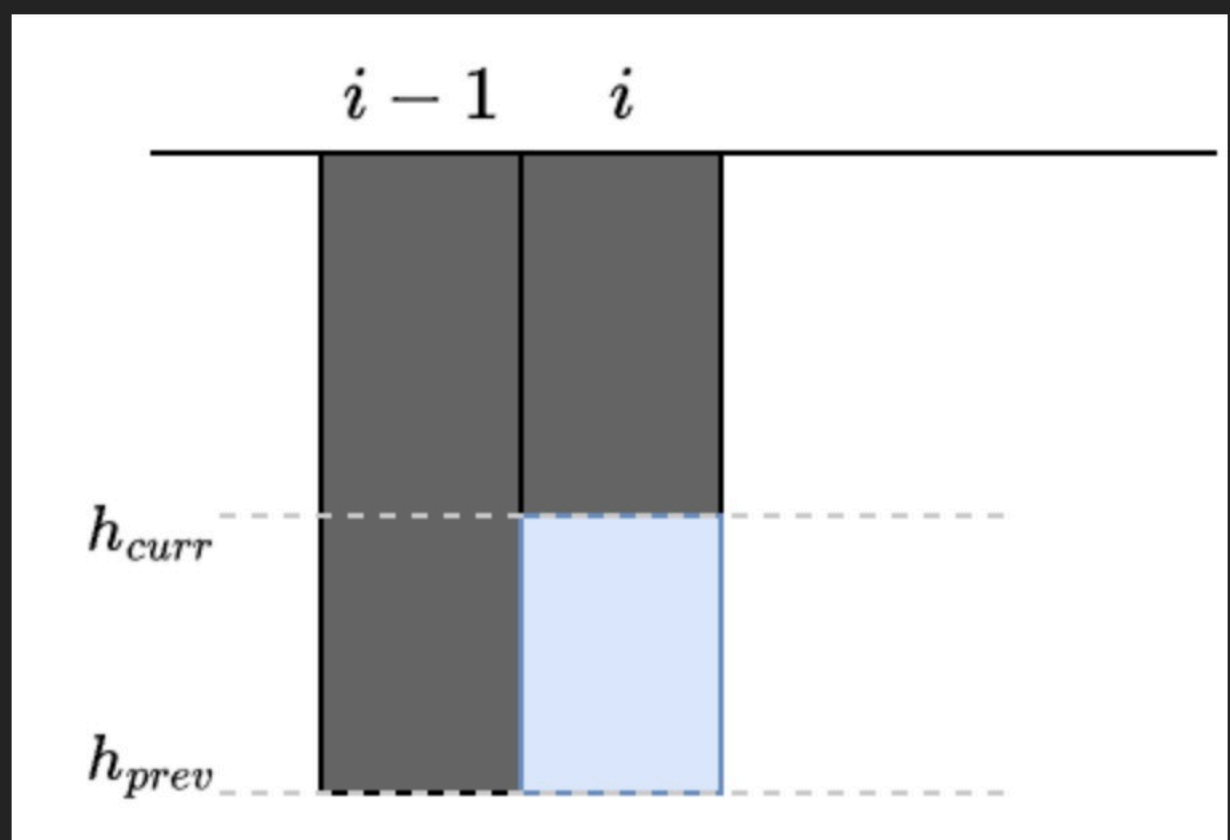
Since the state of each column can be described by a boundary index, and the score of each column depends only on its adjacent columns, dynamic programming is a natural approach for this problem.

We first compute prefix sums for each column as a preprocessing step. Let $S_{i,j}$ denote the sum of the first j elements in the i -th column, where $0 \leq j \leq n$. We define the state $dp[i][h_{curr}][h_{prev}]$ as the maximum score obtained after processing up to the i -th column, where the first h_{curr} elements of column i are black, and the first h_{prev} elements of column

where the first h_{curr} elements of column i are black, and the first h_{prev} elements of column $i - 1$ are black.

The state transition can be derived as follows:

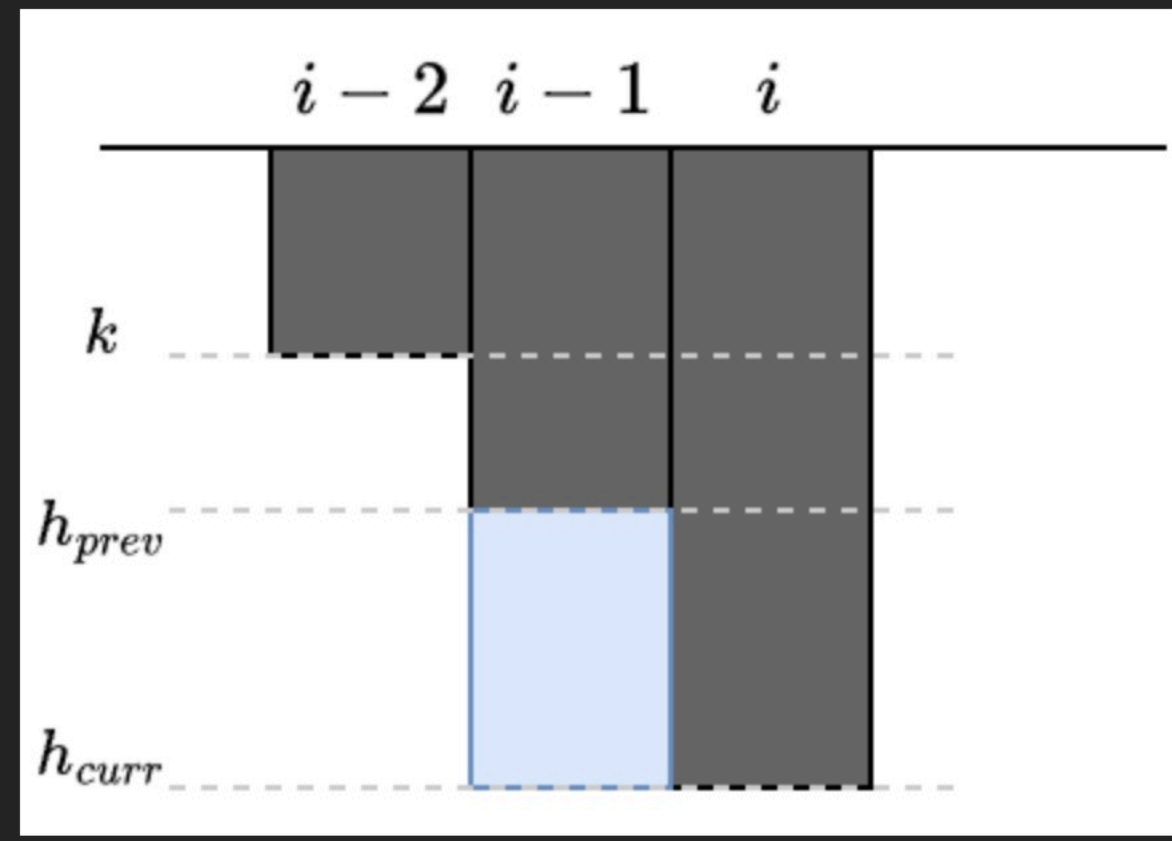
- The state $dp[i]$ captures the coloring of columns i and $i - 1$. This corresponds to fixing h_{curr} in $dp[i - 1]$. Therefore, we enumerate h_{prev} from $dp[i - 1]$, which implicitly enumerates the coloring of column $i - 2$. In this way, we effectively consider three consecutive columns and can compute the resulting score.
- When $h_{curr} \leq h_{prev}$, the current column has fewer black cells than the previous column. In this case, the additional score contributed by the current column must be included, corresponding to the blue region in the figure.



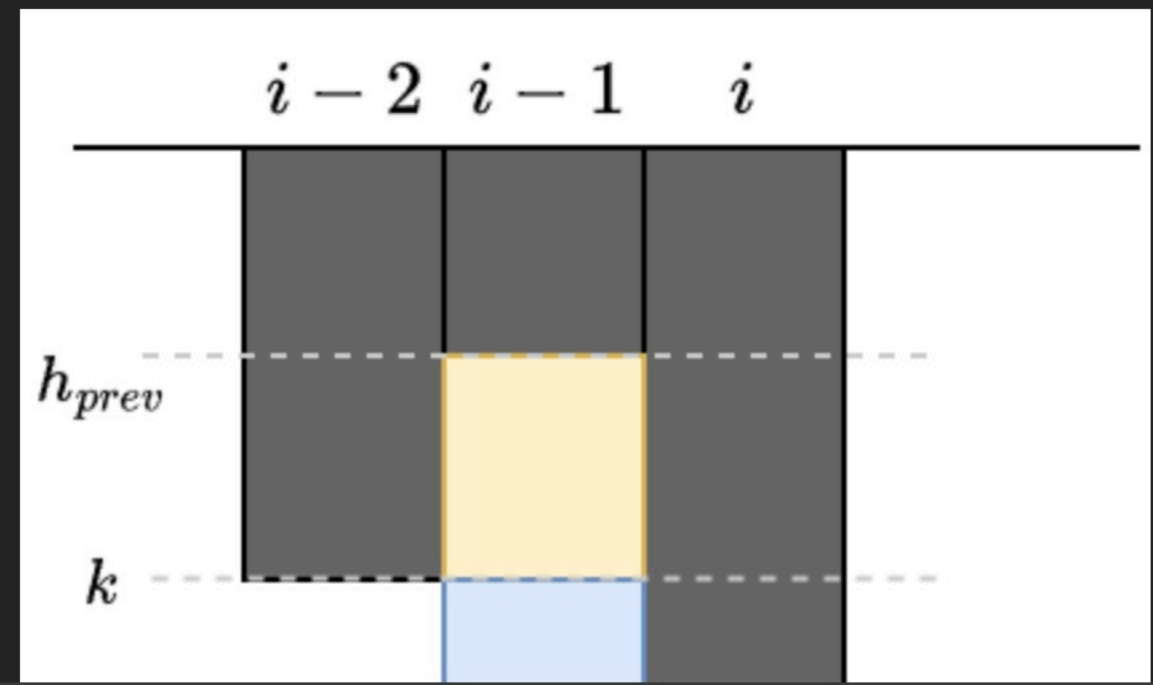
- When $h_{curr} > h_{prev}$, the current column has more black cells than the previous column. Let the previous state's parameter be k . We consider three cases:

Maximum Score From Grid Operations

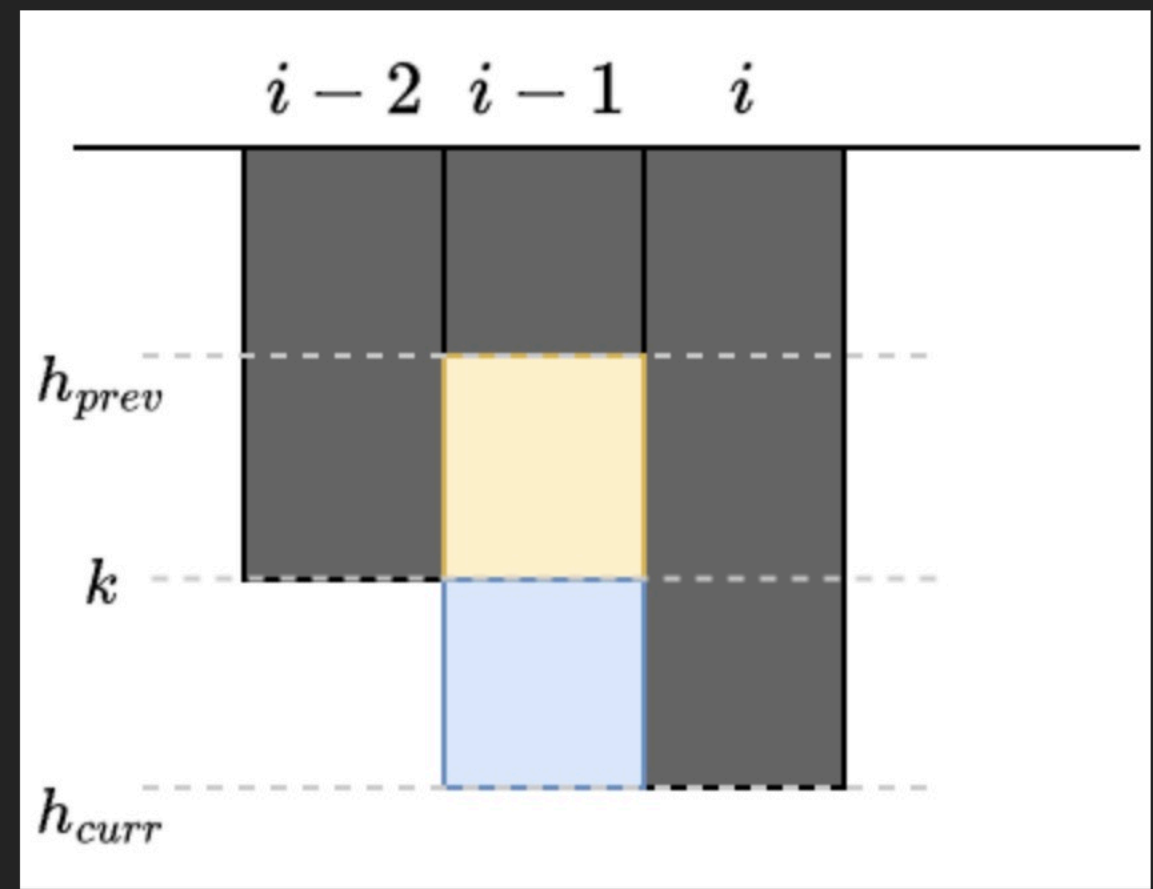
- Case 1: When $k < h_{prev}$, we add the score of the newly covered region introduced by h_{curr} .



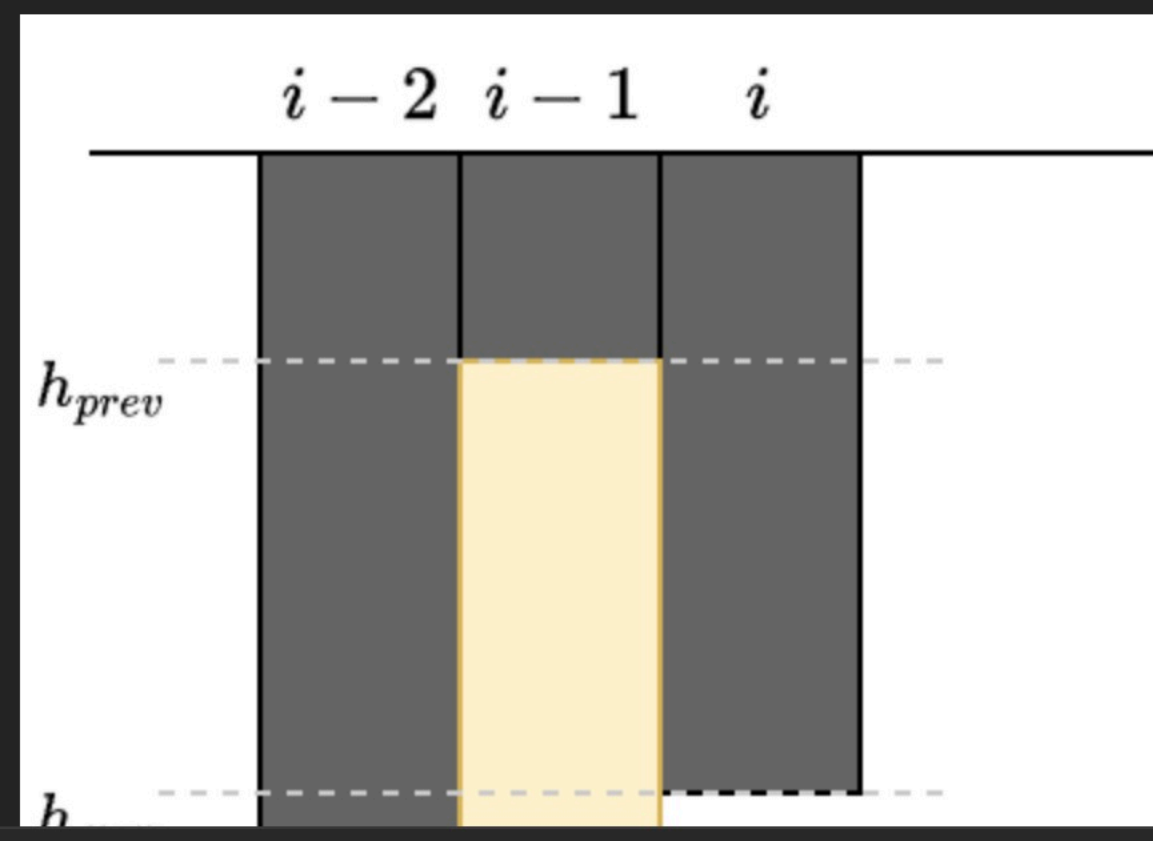
- Case 2: When $h_{prev} \leq k < h_{curr}$, we add only the newly covered region and avoid double counting the already covered part.



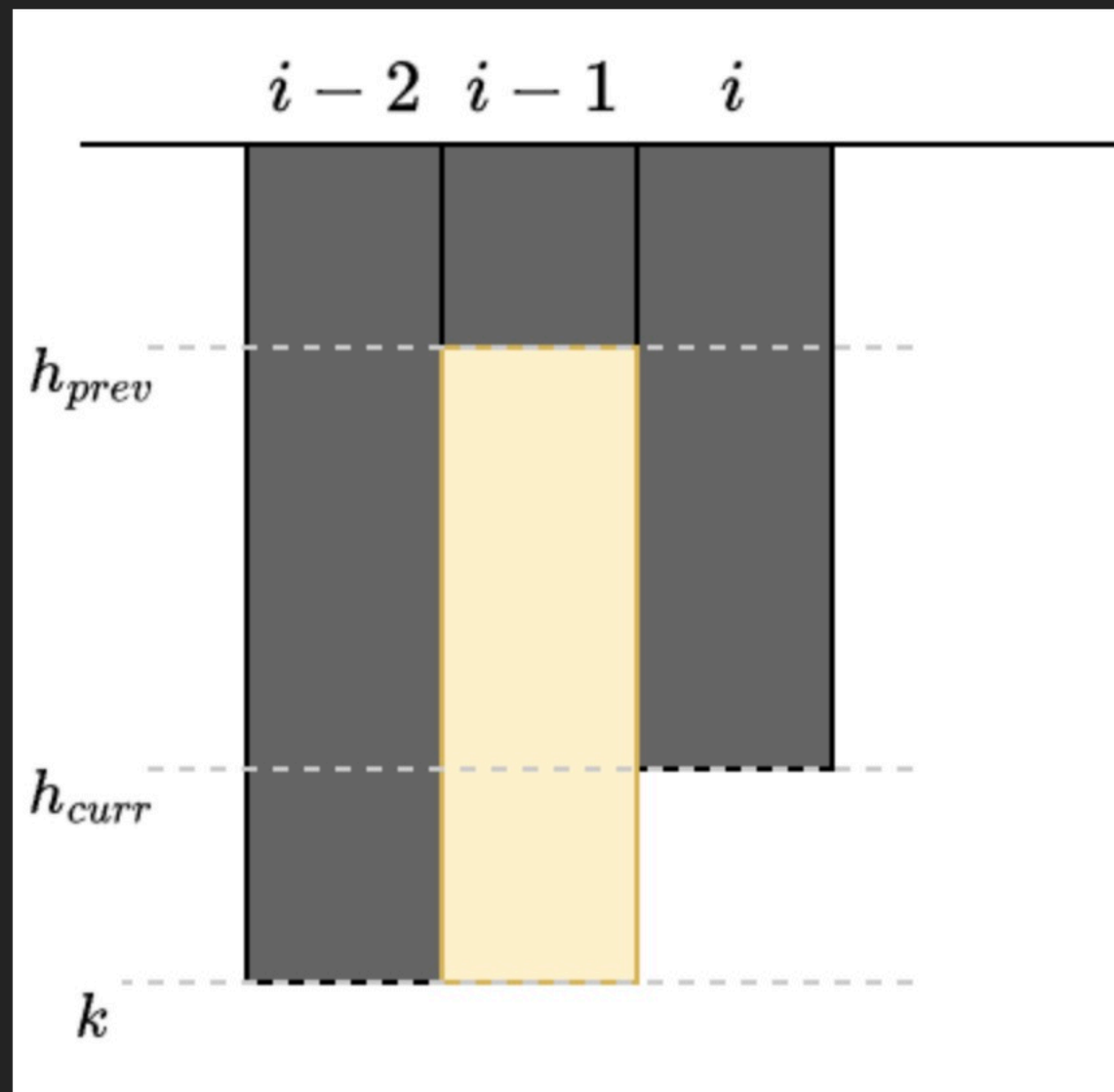
Maximum Score From Grid Operations



- Case 3: When $k \geq h_{curr}$, no new contribution is added, and we directly inherit the previous state.



- Case 3: When $k \geq h_{curr}$, no new contribution is added, and we directly inherit the previous state.

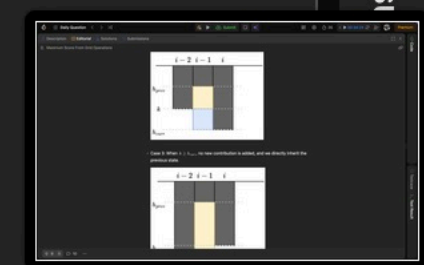


The resulting transition is:

$$dp[i][h_{curr}][h_{prev}] = \begin{cases} \max_{0 \leq k \leq n} \{ dp[i-1][h_{prev}][k] \} + S_{i,h_{prev}} - S_{i,h_{curr}}, & h_{curr} \leq h_{prev} \\ \max_{0 \leq k \leq n} \{ dp[i-1][h_{prev}][k] + \max(0, S_{i-1,h_{curr}} - S_{i-1,\max(h_{prev},k)}) \}, & h_{curr} > h_{prev} \end{cases}$$

Additionally, we need to handle edge case for the first and last columns:

- For the first column, since column 0 does not exist, h_{prev} must be fixed to 0.



📄 Description | 📖 Editorial | 🧪 Solutions | ⌚ Submissions

☰ Maximum Score From Grid Operations 🔗

- For the first column, since column 0 does not exist, h_{prev} must be fixed to 0. Therefore, k can only take the value 0.
- For the last column, the optimal configuration must be either fully black or fully white. This is because changing the last column does not affect any future columns, so only extreme configurations can maximize the score.

Optimizing the h_{prev} state

The above approach has a time complexity of $O(n^4)$, which is too slow. The bottleneck is the enumeration of k . We aim to eliminate this by preprocessing useful maximum values.

When $h_{curr} \leq h_{prev}$, the transition does not depend on k except through:

$$\max_{0 \leq k \leq n} dp[i-1][h_{prev}][k]$$

This allows us to compute the transition in $O(1)$ time if we precompute this maximum.

For the case $h_{curr} > h_{prev}$, we analyze based on whether $k \leq h_{curr}$:

- When $k \leq h_{curr}$, we always add a fixed extra score. To avoid double counting, we maintain a prefix maximum that excludes contributions from column $i-1$:

$$\textit{prevMax}[h_{\textit{prev}}][j] = \max_{0 \leq k \leq j} \left\{ dp[i-1][h_{\textit{prev}}][k] - \max(0, S_{i-1, k} - S_{i-1, h_{\textit{prev}}}) \right\}$$

Now the transition becomes independent of k , and we can update in $O(1)$ time.

- When $k > h_{curr}$, no additional score is added. We only need the suffix maximum:

$$\textit{prevSuffixMax}[h_{prev}][j] = \max_{j \leq k \leq n} dp[i-1][h_{prev}][k]$$

↑ 9 ↓

💬 10 ...

📄 Code

📄 Testcase

📄 Test Result

The optimized transition becomes:

$$dp[i][h_{curr}][h_{prev}] = \begin{cases} prevSuffixMax[h_{prev}][0] + S_{i,h_{prev}} - S_{i,h_{curr}}, & h_{curr} \leq h_{prev} \\ \max (prevSuffixMax[h_{prev}][h_{curr}], prevMax[h_{prev}][h_{curr}]) + S_{i,h_{curr}} - S_{i,h_{prev}}, & h_{curr} > h_{prev} \end{cases}$$

Implementation

C++ Java C C# JavaScript Go Python3 Rust TypeScript  Copy

```
1 using vll = std::vector<long long>;
2
3 class Solution {
4 public:
5     long long maximumScore(vector<vector<int>>& grid) {
6         int n = grid[0].size();
7         if (n == 1) {
8             return 0;
9         }
10
11         vector<vector<vll>> dp(n, vector<vll>(n + 1, vll(n + 1, 0)));
12         vector<vll> prevMax(n + 1, vll(n + 1, 0));
13         vector<vll> prevSuffixMax(n + 1, vll(n + 1, 0));
14         vector<vll> colSum(n, vll(n + 1, 0));
15
16         for (int c = 0; c < n; c++) {
17             for (int r = 1; r <= n; r++) {
18                 colSum[c][r] = colSum[c][r - 1] + grid[r - 1][c];
19             }
20         }
21
22         for (int i = 1; i < n; i++) {
23             for (int currH = 0; currH <= n; currH++) {
24                 for (int prevH = 0; prevH <= n; prevH++) {
25                     if (currH <= prevH) {
26                         long long extraScore =
27                             colSum[i][prevH] - colSum[i][currH];
```

Complexity Analysis

 Description |  Editorial |  Solutions |  Submissions

Maximum Score From Grid Operations

Complexity Analysis

Let n be the dimension of the square matrix $grid$.

- Time complexity: $O(n^3)$.

Computing prefix sums takes $O(n^2)$. Each DP transition, along with prefix and suffix maximum computations, takes $O(n^2)$ per column. Over all columns, this results in $O(n^3)$.

- Space complexity: $O(n^3)$.

The DP table requires $O(n^3)$ space. Additional arrays for prefix sums and auxiliary maximums require $O(n^2)$. Using a rolling array can reduce space to $O(n^2)$.

 Comments (10) Sort by: Best

Type comment here...

Comment

 Article Commenting Rules

1. This comment section is for questions and comments regarding this **LeetCode article**. All posts must respect our [LeetCode Community Rules](#).

2. Concerns about errors or bugs in the article, problem description, or test cases should be posted on [LeetCode Feedback](#), so that our team can address them.

 9 

 10 